Let’s say you have a site that sells some kind of merchandise and you want to track how many users utilize a new feature in your code during the checkout process. You could use the database of orders to sort through your data, but there is usually personal data as well as financial data in that database, which requires special permissions and security procedures to access. By using an arbitrary metric library, you can collect any statistics you want on your consumer’s usage separately from the rest of the application. Because the data pertains to how your collective consumers use the site rather than individual transactions with customer details, the security around it doesn’t have to be as stringent, which opens up the possibility of collecting and analyzing this data in real time.

Another use case is as an indicator of quality—is your application doing what it’s supposed to do? If it is, then you should see your arbitrary metrics behaving as you’d expect. If you start to see those metrics displaying unusual patterns, that may indicate that something isn’t working correctly.

StatsD is designed to write to a server called Graphite (graphite.wikidot.com/faq), a time-data series database with a front end that shows arbitrary charts rather simply. That looks something like figure 6.5.

Figure 6.5. A screenshot of Grafana, the web front end for the Graphite time-series database. Note that the metrics displayed are arbitrary metrics defined in the code of the application.